SOFTWARE DEVELOPMENT WITH PYTHON

Setting Up PostgreSQL

Setup, Installation, and Verification

Module 1 of 3 · Database Fundamentals and Application Development with PostgreSQL
Platforms covered: **Windows 10/11** and **Ubuntu / Debian (Linux)**

Prepared 19 October 2026 · PostgreSQL 18 · psql and VS Code

Contents

What this module is, and what it is not 3

The one idea this module rests on 3

1 Before you begin 3

2 Windows installation 4

2.1 Step 1 — Download the installer 4

2.2 Step 2 — Run the installer and choose components carefully 4

2.3 Step 3 — Set the superuser password, and write it down 4

2.4 Step 4 — Accept the remaining defaults 5

2.5 Step 5 — Verify the service is running 5

2.6 Step 6 — Verify `psql` is available 5

2.7 Step 7 — Connect for the first time 6

2.8 Step 8 — Create a database and connect to it 6

2.9 Step 9 — Create a personal database role 7

3 Linux installation (Ubuntu and Debian) 7

3.1 Step 1 — Ubuntu 8

3.2 Step 2 — Debian 8

3.3 Step 3 — Confirm the server is installed and running 9

3.4 Step 4 — Connect, and understand why this is different from Windows 9

3.5 Step 5 — Confirm the server version from inside 10

3.6 Step 6 — Create a database and a personal role 10

3.7 Step 7 — Confirm the port 10

4 Connecting VS Code to PostgreSQL 11

4.1 Install the correct extension 11

4.2 Add the connection 11

4.3 Confirm the connection works 12

5 Verification checklist 13

5.1 Check 9 deserves emphasis 13

6 Troubleshooting 14

7 Instructor notes 15

8 Module 1 submission 15

What this module is, and what it is not

This module has one job: getting a working PostgreSQL installation onto every student's machine, and proving to them that it works. That is the entire deliverable.

It deliberately contains **no teaching about databases**. What a database is, why dictionaries fail, what a table is, how SQL relates to PostgreSQL — all of that is Module 2. If you find yourself explaining the relational model in this module, stop; you have drifted.

The exception is one idea that the commands themselves require, given in the next section. Every command below depends on it, so students who do not hold it will not be able to troubleshoot their own failures later.

The one idea this module rests on

The one idea this module rests on

> **PostgreSQL is a separate program that runs on your computer, independently of Python and independently of VS Code.** It starts when the machine boots, it keeps running when you close every terminal, and it holds its data in its own files.

Everything in this module follows from that. `psql` is not a program that opens a database — it is a **client** that connects to a database **server** that is already running. If you install PostgreSQL and the server is not running, nothing works, and no amount of reinstalling `psql` will fix it. This one distinction will resolve most of the problems students will hit.

Before you begin

Students should already have:

- A machine with administrator or `sudo` access.
- Python 3 installed and working, which they have from the earlier part of the course.
- VS Code installed. If not, it is a free download from `code.visualstudio.com`.
- About **90 minutes** for this module, most of which is downloading and waiting.

Check disk space too. PostgreSQL needs roughly 300 MB installed on Windows, and if a student has previously installed a different database, there may be an old copy still present that will cause a port conflict later — see the troubleshooting section.

On version

> These instructions target **PostgreSQL 18**, which is the current release and is supported until November 2030. PostgreSQL 17 is acceptable if that is what a student can install; it is supported until November 2029. **PostgreSQL 14**

> **reaches end of life on 12 November 2026**, which is a matter of days from now — if a student has version 14 installed, it must be upgraded before the module is attempted.

Windows installation

Step 1 — Download the installer

Go to `postgresql.org/download/windows`. Note the site's warning: the Windows installer is hosted by EnterpriseDB (EDB), not by the PostgreSQL community servers. That is normal and expected.

Download the **PostgreSQL 18 x86-64 installer**. There is no practical difference between the 32-bit and 64-bit builds for this course — take the 64-bit one.

> **Alternative**
>
> `winget` can install PostgreSQL 18 with `winget install -e --id PostgreSQL.PostgreSQL.18`. Be aware that the package's silent-install mode is unreliable, and the installer still requires you to type a password interactively, so this route still shows the same graphical installer. Students are better served downloading directly.

Step 2 — Run the installer and choose components carefully

This is the step where most Windows students lose time, because the component list is long and the defaults are not what this course needs.

In the component selection screen:

- **Keep** PostgreSQL Server. Obviously.
- **Keep** Command Line Tools. This installs `psql`, which the entire course depends on.
- **Uncheck pgAdmin.** This course uses VS Code as the database tool, not pgAdmin. Installing both wastes disk space and creates a second thing to learn.
- **Uncheck Stack Builder** if it appears. It is a downloader for optional extensions the course does not use.
- Leave the rest alone.

Step 3 — Set the superuser password, and write it down

> **The single most common source of trouble**
>
> The installer asks you to set a password for the `postgres` superuser account. **This is the single most common source of trouble later in this course**, and it happens because the password is typed into an installer dialog, never written down, and forgotten by the next session.
> Each student must choose a password, record it in their own Module 1 notes,

> and be able to reproduce it without checking. Set it now, while the installer is open.
> A class-standard password on locally-installed student machines is acceptable for teaching purposes. If you prefer otherwise, set a personal one — but the requirement that it be written down is not negotiable.

Step 4 — Accept the remaining defaults

- **Data directory** — leave as the default.
- **Port** — leave as **5432**. This is PostgreSQL's conventional port and the course assumes it throughout. If a student changes it, everything in Module 2 and the Python integration that follows will need that changed port, so it is much simpler to leave it alone.
- **Locale** — leave as English.

Click Next, let it install, and finish.

Step 5 — Verify the service is running

Open **PowerShell** (press Windows key, type `powershell`, press Enter). Run:

```
Get-Service postgresql*
```

You should see a service named something like `postgresql-x64-18` with the status `Running`. If it is `Stopped`, start it:

```
Start-Service postgresql-x64-18
```

The name of the service includes the major version, so a student who upgrades will see the name change. If you ever need to manage this service in the GUI, Windows allows it through Services (`services.msc`) — but the PowerShell command is faster and students will be more comfortable with it.

Step 6 — Verify `psql` is available

Still in PowerShell:

```
psql --version
```

You should see something resembling `psql (PostgreSQL) 18.x`.

> If you get a "command not found" or `'psql' is not recognized` error, the installer did not add PostgreSQL's `bin` folder to the system PATH. The full path to the executable is normally:
>
> ```
> C:\Program Files\PostgreSQL\18\bin\psql.exe
> ```

> Run it by full path to confirm the installation is fine, and either add that folder to PATH permanently or simply use the full path for now. **Do not have students reinstall** — the server is installed correctly and only the PATH variable is missing.

Step 7 — Connect for the first time

```
psql -U postgres
```

The `-U` flag names the user to connect as. Enter the password you set in Step 3 when prompted — note that **PowerShell will display nothing as you type**, not even asterisks. This is normal PostgreSQL and Windows behaviour and it unsettles almost every student the first time. Type the password and press Enter.

You should land at a prompt beginning with `postgres=#`.

Confirm the server version from the database's own point of view:

```
SELECT version();
```

Step 8 — Create a database and connect to it

```
CREATE DATABASE school_db;
```

Then move into it:

```
\c school_db
```

> The `\c` command is a **psql meta-command**, not SQL — the backslash is what distinguishes them. `CREATE DATABASE school_db;` is SQL and must end with a semicolon; `\c school_db` is a client-side instruction and must not. Students who confuse these two will produce confusing errors, so name the distinction when it arises.

Confirm where you are:

```
SELECT current_database();
```

The answer should be `school_db`.

Step 9 — Create a personal database role

On Windows, students have so far been connecting as `postgres`, which is the superuser — the account with unrestricted authority. That account is for administration, not for everyday application work. Creating a normal account now, while nothing depends on it, is far easier than retrofitting one later.

Back in the `postgres` database, run:

```
CREATE ROLE student WITH LOGIN CREATEDB PASSWORD 'your_password_here';
```

Use the same password students chose earlier, and remind them to keep the three parts consistent: the Windows password, the `postgres` password, and this one. When these drift apart, the resulting "password authentication failed" error is genuinely hard for beginners to reason about.

Then connect **as that role**, to prove it works:

```
psql -U student -d school_db
```

Success here proves four things at once: the role exists, the password is correct, the database is reachable by name, and authentication is functioning. It is a single command that eliminates most of what can go wrong in Windows setup.

Linux installation (Ubuntu and Debian)

Ubuntu's own software repositories ship an **older** PostgreSQL than the current release. To get version 18 on Ubuntu or Debian, you must add the **PGDG repository** — the official repository maintained by the PostgreSQL project — and install from there. The commands below differ slightly between Ubuntu and Debian because of how each stores repository signing keys.

Read before pasting

> Every command in this section begins with `sudo`, and the student's password will be requested. Teach students to read each command before running it rather than pasting a block blindly.

Step 1 — Ubuntu

Install the prerequisite packages:

```
sudo apt update
sudo apt install -y curl ca-certificates gnupg lsb-release
```

Create the folder that will hold the repository signing key:

```
sudo install -d /usr/share/postgresql-common/pgdg
```

Download the signing key into it:

```
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc https://www.
postgresql.org/media/keys/ACCC4CF8.asc
```

Make the key readable:

```
sudo chmod 644 /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc
```

Add the repository itself:

```
sudo sh -c 'echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.
asc] https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > /etc/apt/
sources.list.d/pgdg.list'
```

The `$(lsb_release -cs)` part automatically expands to the correct codename — `noble` for Ubuntu 24.04, `jammy` for 22.04 — so the same command works across Ubuntu versions. That is worth pointing out, because students who have seen a guide with the codename hard-coded will otherwise assume this command is broken.

Refresh the package list and install:

```
sudo apt update
sudo apt install -y postgresql-18
```

Step 2 — Debian

Debian's official documentation puts the repository in a slightly different location and packages it slightly differently:

```
sudo apt update
sudo apt install -y postgresql-common ca-certificates curl
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc https://www.
postgresql.org/media/keys/ACCC4CF8.asc
sudo chmod 644 /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc
sudo sh -c 'echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.
asc] https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > /etc/apt/
sources.list.d/pgdg.list'
```

```
sudo apt update
sudo apt install -y postgresql-18-pgdg
```

A real difference between the two

> Note the package name: Debian installs **`postgresql-18-pgdg`**, Ubuntu installs **`postgresql-18`**. This is a genuine difference between the two distributions, not a typo, and it is a common source of "no installation candidate" errors when instructions from the wrong distribution are followed.

Step 3 — Confirm the server is installed and running

Check the version:

```
psql --version
```

Check the service:

```
sudo systemctl status postgresql
```

The status line should read `active (running)`. If it does not, start it and set it to start at boot:

```
sudo systemctl enable --now postgresql
```

Verify with `systemctl status postgresql` again.

Step 4 — Connect, and understand why this is different from Windows

This is the part of Linux setup that surprises students most, so explain it rather than moving past it.

Type this:

```
psql
```

It will fail. The error will say something like `FATAL: role "yourname" does not exist`.

Nothing is wrong. On Linux, PostgreSQL uses an authentication method called **peer authentication** for local connections: your operating system username must match the PostgreSQL role name. If the student's system account is `ade`, then connecting as the `postgres` role requires running the command *as the operating-system user called `postgres`*. Windows has no equivalent mechanism, which is why Step 7 of the Windows instructions simply worked there.

Run instead:

```
sudo -u postgres psql
```

Or open a shell as that user and then connect:

```
sudo su - postgres
psql
```

Type `exit` and then `exit` again to leave both shells.

Step 5 — Confirm the server version from inside

```
SELECT version();
```

> Note that `psql --version` reports the version of the **client**, while `SELECT version();` reports the **server**. These can differ, and the server's answer is the authoritative one for the course.

Step 6 — Create a database and a personal role

```
CREATE DATABASE school_db;
CREATE ROLE student WITH LOGIN CREATEDB PASSWORD 'your_password_here';
```

Exit and reconnect as the new role, over a TCP connection, so that the role is genuinely tested rather than inherited from the `sudo -u postgres` shortcut:

```
psql -h localhost -U student -d school_db
```

Enter the password when prompted. This will ask for a password rather than silently using peer authentication, and it should succeed.

Step 7 — Confirm the port

```
sudo ss -ltnp | grep 5432
```

You should see a line showing PostgreSQL listening on port 5432. If this returns nothing, the server is not listening, and the cause is nearly always that the service is stopped.

Connecting VS Code to PostgreSQL

Students will write and manage SQL inside VS Code for the rest of the course, so this connection needs to be established and proven now rather than discovered as a problem in Module 2.

Install the correct extension

Open VS Code, press `Ctrl+Shift+X` to open Extensions, and search for **PostgreSQL**.

Install the extension with the identifier **`ms-ossdata.vscode-pgsql`**.

Do not install the other one

> There is an older extension called `ms-azuretools.vscode-postgresql`, and it also appears in search results. **Do not install it.** It has been deprecated and no longer receives updates. It is the most widely searched and most widely recommended PostgreSQL extension for VS Code, so students will find tutorials pointing at it — expect to correct them.

If a student has the deprecated extension installed from an earlier attempt, uninstall it first, because two PostgreSQL extensions in the same editor will conflict.

Add the connection

Open the Command Palette with `Ctrl+Shift+P`, type `PostgreSQL: Add New Connection`, and press Enter. Complete the connection form:

Field	Value
Server Host	`localhost`
Server Port	`5432`
User Name	`student`
Password	the password the student recorded in Step 7 of their platform's instructions
Database	`school_db`
Save password	Enable, and store it in VS Code's encrypted credential storage, **not** in `settings.json`
SSL Mode	`Disable` is acceptable for local course work

The equivalent connection string, which students will meet later when connecting from Python, is:

```
postgresql://student:password@localhost:5432/school_db
```

Note the order: scheme, username, password, host, port, database name. Students will need to construct this themselves in Module 3, so it is worth writing it out and asking them to read the structure rather than copy it.

Keep the password out of settings.json

Do not put the password directly into `settings.json`. The extension's default storage is encrypted in the operating system's credential store, and students who override it will be storing a database password in plain text in a file they will eventually commit to a repository. This is a small habit that prevents a real problem later, and it costs thirty seconds now.

Confirm the connection works

In the Explorer sidebar, expand the **PostgreSQL** panel. Students should see their `school_db` database, and inside it a **Schemas** entry, and inside that a **Tables** folder.

If the tree populates, the connection is established and the database is visible from within the editor.

Test it end to end by opening a new query file (`Ctrl+N`), typing `SELECT version();`, and running it with the play button that appears above the query. A result grid should appear in the lower panel.

Verification checklist

This is the deliverable of Module 1. Students must complete every check and retain the evidence.

Each item states the command to run and what success looks like. Students should capture output as text in their Module 1 notes or as a screenshot — either is acceptable, but the output must be legible.

#	Check	Command	Success looks like
1	Server version is 18	`SELECT version();`	Output contains `PostgreSQL 18.`
2	The service is running	`systemctl status postgresql` (Linux) `Get-Service postgresql*` (Windows)	`active (running)` `Running`
3	Connect as superuser	`sudo -u postgres psql` `psql -U postgres` (Windows)	Prompt appears ending in `=#`
4	Your database exists	`\l`	`school_db` appears in the list
5	You can enter it	`\c school_db`	Prompt now reads `school_db=#`
6	Your role works	`psql -U student -d school_db` (Windows) `psql -h localhost -U student -d school_db` (Linux)	Connects without error
7	Create and query a table	`CREATE TABLE test (id integer);` then `INSERT INTO test VALUES (1);` then `SELECT * FROM test;`	One row returned
8	VS Code connects	Explorer → PostgreSQL panel	`school_db` visible with an expandable tree
9	Data survives a restart	See below	Row still present

Check 9 deserves emphasis

It is the one that proves the student has a database rather than a scratchpad, and it is a five-minute exercise that pays for itself every session afterwards.

On Linux: `sudo systemctl restart postgresql`, then reconnect and run `SELECT * FROM test;` — the row is still there.

On Windows: restart the PostgreSQL service in PowerShell, then reconnect and run the same query.

Tell students what this demonstrates before they run it. Throughout the course they will be replacing dictionary storage, and the first thing that needs to be understood is that the data lives in a program that was never stopped.

Students should also clean up the test table afterwards, or leave it — it is a good opportunity to demonstrate the `DROP TABLE` command, which is SQL and therefore belongs to Module 2. Do not teach it here; simply do not be surprised if a student already knows it.

Troubleshooting

Nearly every problem students hit falls into this table. Teach them to read the error message rather than to search for a solution to it, because searching for the error leads them to forum posts describing a different system.

Symptom	Cause	Fix
`psql: command not found`	Not installed, or PATH not updated	`sudo apt install postgresql-18`; open a new terminal after installing
`'psql' is not recognized`	Installer did not add `bin` to PATH	Use the full path `C:\Program Files\PostgreSQL\18\bin\psql.exe`
`FATAL: role "xxx" does not exist`	Peer authentication — OS user must match role	Use `sudo -u postgres psql`
`password authentication failed`	Wrong password, or password never recorded	Recheck Module 1 notes; on Windows confirm the installer password; on Linux reset as the `postgres` user
`Nothing appears when a password is typed`	Normal — input is hidden	Type it anyway and press Enter
`connection refused`	Server is not running	`sudo systemctl enable --now postgresql` `Start-Service postgresql-x64-18`
`port 5432 is already in use`	Another PostgreSQL, Docker container, or old install holds the port	Find it with `sudo ss -ltnp | grep 5432`; stop the old service or uninstall it
`E: Unable to locate package postgresql-18`	PGDG repository not added, or you followed Debian steps on Ubuntu	Re-run the correct platform's repository steps; check that `$(lsb_release -cs)` expanded
`E: Unable to locate package postgresql-18-pgdg`	You are on Ubuntu, not Debian	Ubuntu uses `postgresql-18`, without the `-pgdg` suffix
`VS Code: self-signed certificate error`	Local server uses its own certificate	Set SSL Mode to `Disable` for local work
`VS Code: extension fails to load, GLIBC error`	Very old Linux distribution	Update the distribution; the extension needs a modern base
`CREATE DATABASE fails with ``already exists''`	It was already created	Drop it with `DROP DATABASE school_db;` or just connect to it
`Second CREATE ROLE student fails`	Role already exists	Drop it with `DROP ROLE student;` then recreate

A deliberate inconsistency

`DROP TABLE` and `DROP DATABASE` appear in the table above as remedies, which is a small deliberate inconsistency. This module does not teach SQL. Students who copy a remedy without understanding it are acting exactly as this course is designed to prevent. Frame every line here as something to try *now*, with the reasoning to follow in Module 2 — not as a reference they may draw on later.

Instructor notes

Rehearse on your own machine first. Run the full sequence on a clean Windows machine and a clean Ubuntu machine before the session. Almost every problem in the table above is one you will have already seen and solved once, and it is worth fifteen minutes to be the person who has seen them.

Two problems will account for most of your support time. The first is the forgotten superuser password on Windows. The second, and more common, is the peer authentication error on Linux when a student types `psql` instead of `sudo -u postgres psql`. Neither is difficult to fix; both are alarming to a beginner who has never administered anything. Say the first one out loud in the session so students know to expect it.

Resist teaching theory. The pull to explain the relational model while setting up is strong and it is misplaced. Module 2 exists for that. If a student asks what a database is during setup, the honest answer is one sentence — *"we'll cover that next session"* — and the time saved compounds across the rest of the module.

Make check 9 a class event. Restarting the server and finding the data intact is the cheapest, most convincing demonstration available, and it sets up the argument for the rest of the course. Do it live on the projector.

Decide your Windows password policy deliberately. Either a class-standard password or individual ones, both work. What does not work is leaving it undecided, because that produces a class where a third of the students cannot connect in Module 2 and the reason is unknown to everyone including them.

Module 1 submission

Students submit **evidence of all nine verification checks** plus a short statement of: their operating system and version, their installed PostgreSQL version, their operating system username, the PostgreSQL role name they created, and the port in use.

The statement is not paperwork. Naming these five facts in order is the same diagnostic a developer performs when something is not connecting, and it makes the difference between a student who knows their own environment and one who does not.

What this module does not cover, by design

> What a database is, relational versus non-relational, PostgreSQL's history, SQL syntax of any kind, table design, or how the application will eventually connect. All of that is Module 2.

Sources

postgresql.org/download · postgresql.org/download/windows · postgresql.org/download/linux/ubuntu · postgresql.org/download/linux/debian · postgresql.org/support/versioning · marketplace.visualstudio.com (ms-ossdata.vscode-pgsql) · github.com/microsoft/vscode-pgsql · enterprisedb.com/docs/dev-guides/deploy/windows